

Caso práctico 1

En el siguiente ejemplo se borra el departamento número 20, pero evitando posibles errores por violar restricciones de integridad referencial, pues el departamento tiene empleados asociados. Para ello crearemos antes un departamento provisional, al que asignamos los empleados del departamento 20 antes de borrar dicho departamento. El programa también informa del número de empleados afectados.

Antes de introducir el ejemplo deberemos introducir la instrucción SET SERVEROUTPUT ON para que muestre los mensajes.

```
DECLARE
v_num_empleados NUMBER(2);
BEGIN
INSERT INTO depart VALUES (99,'PROVISIONAL',NULL);
UPDATE emple SET dept_no = 99
WHERE dept_no = 20;
v_num_empleados := SQL%ROWCOUNT;
DELETE FROM depart
WHERE dept_no = 20;
DBMS_OUTPUT.PUT_LINE(v_num_empleados ||
' Empleados ubicados en PROVISIONAL');
EXCEPTION
WHEN OTHERS THEN
RAISE_APPLICATION_ERROR(-20000, 'Error en aplicación');
END;
/
```

El resultado de la ejecución del programa será:

5 Empleados ubicados en PROVISIONAL

Procedimiento PL/SQL terminado con éxito.

Caso práctico 2

El siguiente bloque visualiza el apellido y el oficio del empleado cuyo número es 7900.

```
DECLARE
v_ape VARCHAR2(10);
v_oficio VARCHAR2(10);
BEGIN
SELECT apellido, oficio INTO v_ape, v_oficio
FROM EMPL WHERE EMP_NO = 7900;
DBMS_OUTPUT.PUT_LINE(v_ape||'*'||v_oficio);
END;
/
```

El resultado de la ejecución del programa será:

JIMENO*EMPLEADO

Procedimiento PL/SQL terminado con éxito.

Caso práctico 3

El ejemplo anterior, con gestión de excepciones, sería:

```
DECLARE
v_ape VARCHAR2(10);
v_oficio VARCHAR2(10);
BEGIN
SELECT apellido, oficio INTO v_ape, v_oficio
FROM EMPL WHERE EMP_NO = 7900;
DBMS_OUTPUT.PUT_LINE(v_ape||'*'||v_oficio);

EXCEPTION
WHEN NO_DATA_FOUND THEN
RAISE_APPLICATION_ERROR(-20000,'ERROR no hay datos');
WHEN TOO_MANY_ROWS THEN
RAISE_APPLICATION_ERROR(-20000,'ERROR demasiados datos');
WHEN OTHERS THEN
RAISE_APPLICATION_ERROR(-20000,'Error en la aplicación');
END;
```

Caso práctico 4

El siguiente código crea un trigger que se ejecutará automáticamente cuando se elimine algún empleado en la tabla correspondiente visualizando el número y el nombre de los empleados borrados:

```
CREATE OR REPLACE TRIGGER audit_borrado_emple
BEFORE DELETE
ON emple
FOR EACH ROW
BEGIN
DBMS_OUTPUT.PUT_LINE('BORRADO EMPLEADO'
|| '* ' ||:old.emp_no
|| '* ' ||:old.apellido);
END;
```

Caso práctico 5

Suponemos la siguiente tabla en la base de datos:

```
CREATE TABLE CLIENTES (  
    CLIENTE_NO    NUMBER(3)    CONSTRAINT    CLIENTES_NUMERO_PK  
    PRIMARY KEY,  
    NOMBRE    VARCHAR2(30)  
);
```

Para la que tenemos los siguientes datos insertados:

```
INSERT INTO CLIENTES VALUES (102, 'MARTA MARTÍNEZ MARTOS');  
INSERT INTO CLIENTES VALUES (001, 'ANA ALONSO ÁLVAREZ');  
INSERT INTO CLIENTES VALUES (600, 'MARCOS MÁRQUEZ MARÍN');  
INSERT INTO CLIENTES VALUES (303, 'ALONSO ALONSO ALONSO');  
INSERT INTO CLIENTES VALUES (99, 'MARTÍN RODRIGO RUIZ');  
INSERT INTO CLIENTES VALUES (55, 'REMEDIOS RETUERTA RAMÍREZ');
```

El siguiente programa solicitará la introducción de un número de cliente y visualizará el nombre del cliente correspondiente con el número introducido. Para introducir el número de cliente recurriremos a las variables de sustitución.

```
DECLARE  
    v_nom CLIENTES.NOMBRE%TYPE; --(ejemplo uso %TYPE)  
BEGIN  
    SELECT nombre INTO v_nom  
    FROM clientes  
    WHERE CLIENTE_NO=&vn_cli;  
    DBMS_OUTPUT.PUT_LINE(v_nom);  
END;  
/
```

Al ejecutar el programa, éste solicitará un valor para la variable de sustitución, y una vez introducido el valor sustituirá a la variable y se enviará el bloque al servidor para su ejecución.

Caso práctico 6

Introduciendo estas líneas desde el indicador de SQL*Plus dispondremos de un procedimiento PL/SQL sencillo para consultar los datos de un cliente:

```
CREATE OR REPLACE
PROCEDURE ver_depart (numdepart NUMBER)
AS
    v_dnombre VARCHAR2(14);
    v_localidad VARCHAR2(14);
BEGIN
    SELECT dnombre, loc INTO v_dnombre, v_localidad
    FROM depart
    WHERE dept_no = numdepart;
    DBMS_OUTPUT.PUT_LINE('Num depart: ' || numdepart || ' *
Nombre dep: ' || v_dnombre ||
' * Localidad: ' || v_localidad);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No encontrado
departamento ');
END ver_depart;
/
Procedimiento creado.
```

Observa:

- La cabecera del procedimiento indica el nombre del mismo y el parámetro que se utilizará para pasarle valores.
- La palabra AS especifica el comienzo del cuerpo del programa y del bloque, comenzando con la zona de declaraciones donde se indicarán los objetos que intervendrán en el programa: variables, constantes, etcétera.
- La palabra BEGIN indica el comienzo de la zona de instrucciones.
- La instrucción SELECT deposita el resultado de la consulta en las variables v_dnombre, v_localidad que siguen a INTO tal como se vio al hablar del soporte para consultas del lenguaje. La cláusula WHERE contiene el parámetro con el que se llamó al programa, que será el número de departamento cuyos datos se requieren.
- El procedimiento DBMS_OUTPUT.PUT_LINE visualiza el contenido de las variables y los literales. Para que funcione correctamente la variable **SERVEROUTPUT debe estar en ON** antes de ejecutar el programa.
- La palabra EXCEPTION indica el comienzo de la zona de tratamiento de excepciones. Esta zona solo se ejecutará si se produce alguna excepción. Por ejemplo, si al ejecutarse la cláusula SELECT no se encuentra ninguna fila que cumpla la condición, se levantará la excepción NO_DATA_FOUND, se bifurcará el control del programa a esta sección.
- El manejador WHEN NO_DATA_FOUND <<caza>> la excepción, en el caso de que se produzca, y ejecuta las instrucciones que siguen a la cláusula THEN dando por finalizado el programa.
- El símbolo <</>> indica a SQL*Plus el final del procedimiento.

Si el compilador detecta errores veremos un mensaje indicándolo.

La orden **SHOW ERRORS** permite ver los errores detectados.

Ahora, el procedimiento se encuentra almacenado en el servidor de la base de datos y puede ser invocado desde cualquier estación por cualquier usuario autorizado y con cualquier herramienta; por ejemplo, desde SQL*Plus mediante la orden **EXECUTE**:

```
SET SERVEROUTPUT ON
EXECUTE ver_depart(20)
Num depart:20 * Nombre dep: INVESTIGACION * Localidad:
MADRID
```

También podemos **invocar al procedimiento desde un bloque PL/SQL**, por ejemplo, de esta forma:

```
BEGIN ver_depart(30); END;
/
Num depart:30 * Nombre dep: VENTAS * Localidad: BARCELONA
```

En realidad, la orden EXECUTE es una utilidad de SQL*Plus que crea un bloque PL/SQL anónimo añadiendo un BEGIN y un END al principio y al final, respectivamente, de la llamada al procedimiento.

Caso práctico 7

Ejemplos de aplicación:

1. En el siguiente procedimiento se visualiza el precio de un producto cuyo número se pasa como parámetro.

```
CREATE OR REPLACE
PROCEDURE ver_precio(v_num_producto VARCHAR2)
AS
v_precio NUMBER;
BEGIN
SELECT precioventa INTO v_precio
FROM productos
WHERE codigoproducto = v_num_producto;
DBMS_OUTPUT.PUT_LINE('Precio = '||v_precio);
END;
/
```

Procedimiento creado.

```
EXECUTE VER_PRECIO('FR-15');
Precio = 25
Procedimiento PL/SQL terminado con éxito.
```

2. Escribiremos un procedimiento que modifique el precio de un producto pasándole el número del producto y el nuevo precio.

El procedimiento comprobará que la variación de precio no supere el 20 por 100:

```
CREATE OR REPLACE
PROCEDURE modificar_precio_producto (numproducto VARCHAR2,
nuevoprecio NUMBER)
AS
v_precioant NUMBER(15,2);
BEGIN
SELECT precioventa INTO v_precioant
```

```

        FROM productos
        WHERE codigoproducto = numproducto;

    IF (v_precioant * 0.20) > (nuevoprecio - v_precioant) THEN
        UPDATE productos SET precioventa = nuevoprecio
        WHERE codigoproducto = numproducto;
    ELSE
        DBMS_OUTPUT.PUT_LINE('Error, modificación supera
20%');
    END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No encontrado producto ' ||
numproducto);
END modificar_precio_producto;
/

```

Ejemplos de ejecución:

```

SET SERVEROUTPUT ON;
EXECUTE MODIFICAR_PRECIO_PRODUCTO('FR-15',27);
Procedimiento PL/SQL terminado con éxito.

SELECT PRECIOVENTA FROM PRODUCTOS WHERE CODIGOPRODUCTO='FR-15';

EXECUTE MODIFICAR_PRECIO_PRODUCTO('FR-15',300)
Error, modificación supera 20%
Procedimiento PL/SQL terminado con éxito.

```

Observamos que en el segundo caso no se ha producido la modificación deseada. Aun así, el procedimiento ha terminado con éxito, ya que no se ha producido ningún error no tratado.

3. Escribiremos una función que devuelva el valor con IVA de una cantidad que se pasará como primer parámetro. La función también podrá recoger un segundo parámetro opcional, que será el tipo de IVA siendo el valor por defecto 16.

```

CREATE OR REPLACE FUNCTION con_iva (cantidad NUMBER, tipo NUMBER
DEFAULT 16)

RETURN NUMBER
AS
    v_resultado NUMBER (10,2) DEFAULT 0;
BEGIN
    v_resultado := cantidad * (1 + (tipo / 100));
    RETURN(v_resultado);
END con_iva;
/

```

Ahora podemos usar la función creada para realizar cálculos dentro de un bloque o programa PL/SQL:

```

BEGIN DBMS_OUTPUT.PUT_LINE(con_iva(200)); END;
/
232

```

Debemos recordar que hay que HACER ALGO con el valor devuelto por la función: visualizarlo, usarlo como parte de una expresión, etcétera. También podemos usarla en instrucciones SQL:

```
SELECT codigoproducto, precioventa, con_iva(precioventa) FROM
productos;
```